Python Fundamentals

A practical, beginner-friendly guide for GenAI developers

Beginner-friendly study guide • Concepts, examples, and practical exercises

1. What Python is

Python is a general-purpose programming language. It is popular in automation, backend services, data analysis, machine learning and GenAI because its syntax is readable and its ecosystem is large.

Interpreter: the program that executes Python code. Script: a file such as main.py. Library/package: reusable code that you install or import.

```
print('Hello, GenAI!')
```

This asks Python to display the text inside the parentheses. Text inside quotes is a string. print is a built-in function.

2. Variables and data types

Variables are names that refer to values. Python assigns a type to each value at runtime; you usually do not declare the type first.

```
name = 'Kethan'\nage = 25\nheight_m = 1.75\nis_learning = True\n
```

• str: text, such as 'hello'.

• int: whole numbers, such as 25.

• float: decimal numbers, such as 1.75.

• bool: True or False.

• None: represents the absence of a value.

Use type(value) to inspect a value's type. Variable names should describe their purpose.

3. Collections

List

A list is ordered and mutable (you can change it). Use it for a sequence that may change.

```
tools = ['Python', 'Transformers', 'Gradio']\ntools.append('FAISS')\nprint(tools[0])
```

Tuple

A tuple is ordered but normally used as an immutable grouping of values.

```
point = (10, 20)
```

Dictionary

A dictionary stores key-value pairs. It is useful for settings, API responses and structured records.

```
user = {'name': 'Kethan', 'role': 'GenAI learner'}\nprint(user['role'])
```

Set

A set stores unique values and is useful when duplicates should be removed.

```
unique_tools = set(['Python', 'Python', 'RAG'])
```

4. Operators and decisions

Operators perform calculations or comparisons. Examples: +, -, *, /, ==, !=, >, <, and, or, not.

```
score = 78\nif score >= 80:\n    print('Excellent')\nelif score >= 50:\n    print('Pass')\nelse:\n    print('Tr
```

if tests a condition, elif tests another condition when earlier ones fail, and else handles remaining cases. Indentation defines the code block.

5. Loops

A for loop visits each item in a sequence. A while loop repeats while a condition remains true. Loops automate repetitive work.

```
documents = ['guide.pdf', 'notes.txt', 'manual.docx']\nfor document in documents:\n    print('Processing:', doc
```

Be careful with while loops: make sure something eventually makes the condition false, otherwise the loop may never stop.

6. Functions

A function is a reusable block of code. Parameters are inputs; return sends a result back to the caller.

```
def chunk_text(text, chunk_size=500):\n    return [text[i:i + chunk_size] for i in range(0, len(text), chunk_si
```

def defines the function. text and chunk_size are parameters. The second parameter has a default value. return gives the result to the caller.

7. Modules, packages and environments

A module is usually a Python file. A package groups modules. import makes code available to your program. A virtual environment isolates a project's installed packages from other projects.

```
python -m venv .venv\n.venv\\Scripts\\activate\npython -m pip install requests
```

On Windows CMD, activate with .venv\\Scripts\\activate. Use python -m pip so pip belongs to the selected Python interpreter. Keep a requirements.txt file to record dependencies.

8. Files, exceptions and JSON

File handling lets an application read documents and save results. Use a context manager (with) so files close reliably. Exceptions let you handle errors without abruptly crashing.

```
from pathlib import Path\nimport json\n\ntext = Path('notes.txt').read_text(encoding='utf-8')\ndata = {'filenam
```

```
try:\n    number = int('not a number')\nexcept ValueError as error:\n    print('Could not convert:', error)
```

JSON is a common text format for structured data and API communication. Never assume a file exists or contains valid data; validate and handle expected errors.

9. Classes and object-oriented programming

A class defines a kind of object; an object is an instance of a class. Attributes store state, while methods define behavior. OOP helps organize larger applications.

```
class Document:\n    def __init__(self, name, text):\n        self.name = name\n        self.text = text\n\n
```

self refers to the current object. __init__ initializes it. You do not need to use classes for every small script; use them when they make related state and behavior easier to manage.

10. HTTP, APIs and environment variables

An API lets programs communicate. HTTP requests commonly use GET to retrieve data and POST to submit data. A response includes a status code and usually a body such as JSON.

```
import os\napi_key = os.getenv('OPENAI_API_KEY')\nif not api_key:\n    raise RuntimeError('Set OPENAI_API_KEY b
```

Environment variables keep secrets outside source code. Never paste API keys into public repositories, screenshots or committed .env files. A .gitignore file should exclude secrets and virtual environments.

11. Python for GenAI

Common tasks include loading model libraries, reading files, cleaning text, calling APIs, splitting documents into chunks, creating embeddings, searching vectors, building a UI and testing outputs.

```
def make_prompt(question, context):\n    return (\n        'Answer using only the context below. If missing, sa
```

Do not treat a prompt as a security boundary. Retrieved documents can contain misleading instructions. Your application should separate trusted instructions from untrusted document text and test prompt-injection cases.

12. Practice tasks

- Write a script that counts words in a .txt file.
- Read a JSON file and print selected fields.
- Create a function that splits text into overlapping chunks.
- Write a try/except block for a missing file.
- Create a small command-line Q&A loop.
- Add logging and tests to a document-processing function.

Suggested order: master variables, collections, conditions, loops, functions and files before moving to classes, APIs and GenAI frameworks.

Official documentation and further reading

Official Python Tutorial — syntax, data structures, functions and modules
<link href='https://docs.python.org/3/tutorial/' color='blue'>https://docs.python.org/3/tutorial/</link>

Python Beginner's Guide — setup and beginner learning paths
<link href='https://wiki.python.org/python/BeginnersGuide.html' color='blue'>https://wiki.python.org/python/BeginnersGuide.html</link>

Python Packaging User Guide — environments, installing packages and packaging
<link href='https://packaging.python.org/' color='blue'>https://packaging.python.org/</link>